1. Visão geral

O Java 8 introduziu novas APIs para *Data* e *Hora* para solucionar as deficiências das antigas classes *java.util.Date* e *java.util.Calendar* .

Neste tutorial, vamos começar com os problemas nas APIs *de Data* e *Calendário* existentes e discutir como as novasAPIs *de Data* e *Hora do Java 8 os resolvem.*

Analisaremos também algumas das classes principais do novo projeto Java 8 que fazem parte do pacote *java.time* , como *LocalDate* , *LocalTime* , *LocalDateTime* , *ZonedDateTime* , *Period* , *Duration* e suas APIs suportadas.

Leitura complementar:

Trabalhando com parâmetros de data no Spring

Aprenda como trabalhar com parâmetros de data no Spring MVC.

Leia mais →

Verificar se uma string é uma data válida em Java

Veja diferentes maneiras de verificar se uma String é uma data válida em Java.

Leia mais →

2. Problemas com as APIs *de data / hora* existentes

- **Segurança de threads** – As classes *Date* e *Calendar* não são thread-safe, o que obriga os desenvolvedores a lidar com a complexidade de problemas de concorrência difíceis de depurar e a escrever código adicional para garantir a segurança de threads. Em contrapartida, as novas APIs *de Date* e *Time* introduzidas no Java 8 são imutáveis e thread-safe, eliminando assim essa preocupação com a concorrência.
- **Design e facilidade de uso da API** – As APIs *de Data* e *Calendário* são mal projetadas, com métodos inadequados para realizar operações cotidianas. A nova API *de Data / Hora* é centrada na ISO e segue modelos de domínio consistentes para data, hora, duração e períodos. Há uma ampla variedade de métodos utilitários que suportam as operações mais comuns.
- ***Data* e *hora*** com fuso horário – Os desenvolvedores precisavam escrever lógica adicional para lidar com o fuso horário nas APIs antigas, enquanto que com as novas APIs, o gerenciamento do fuso horário pode ser feito com*as APIs Local* e*ZonedDate /Time* .

3. Utilizando *LocalDate* , *LocalTime* e *LocalDateTime*

As classes mais comumente usadas são *LocalDate* , *LocalTime* e *LocalDateTime* . Como seus nomes indicam, elas representam a data/hora local do contexto do observador.

Utilizamos essas classes principalmente quando não é necessário especificar explicitamente os fusos horários no contexto. Nesta seção, abordaremos as APIs mais comuns.

3.1. Trabalhando com *LocalDate*

O *LocalDate* representa **uma data no formato ISO (aaaa-MM-dd) sem a hora.** Podemos usá-lo para armazenar datas como aniversários e dias de pagamento.

É possível criar uma instância da data atual a partir do relógio do sistema:

```
LocalDate localDate = LocalDate.now();Cópia
```

E podemos obter o *LocalDate* que representa um dia, mês e ano específicos usando o método *`of`* ou o método *`parse`* .

Por exemplo, estes trechos de código representam a *data local (LocalDate)* para 20 de fevereiro de 2015:

```
LocalDate.of(2015, 02, 20);

LocalDate.parse("2015-02-20");

Cópia
```

O *LocalDate* fornece vários métodos utilitários para obter uma variedade de informações. Vamos dar uma olhada rápida em alguns desses métodos da API.

O seguinte trecho de código obtém a data local atual e adiciona um dia:

```
LocalDate tomorrow = LocalDate.now().plusDays(1);Cópia
```

Este exemplo obtém a data atual e subtrai um mês. Observe como ele aceita um *enum* como unidade de tempo:

```
LocalDate previousMonthSameDay = LocalDate.now().minus(1,
ChronoUnit.MONTHS);Cópia
```

Nos dois exemplos de código a seguir, analisamos a data “2016-06-12” e obtemos o dia da semana e o dia do mês, respectivamente. Observe os valores de retorno — o primeiro é um objeto representando o *DayOfWeek* , enquanto o segundo é um *inteiro* representando o valor ordinal do mês:

```
DayOfWeek sunday = LocalDate.parse("2016-06-12").getDayOfWeek();

int twelve = LocalDate.parse("2016-06-12").getDayOfMonth();

Cópia
```

Podemos verificar se uma data ocorre em um ano bissexto, por exemplo, a data atual:

```
boolean leapYear = LocalDate.now().isLeapYear();Cópia
```

Além disso, a relação entre uma data e outra pode ser determinada como ocorrendo antes ou depois de outra data:

```
boolean notBefore = LocalDate.parse("2016-06-12")
  .isBefore(LocalDate.parse("2016-06-11"));

boolean isAfter = LocalDate.parse("2016-06-12")
  .isAfter(LocalDate.parse("2016-06-11"));

Cópia
```

Por fim, os limites de data podem ser obtidos a partir de uma data específica.

Nos dois exemplos a seguir, obtemos o *LocalDateTime* que representa o início do dia (2016-06-12T00:00) da data fornecida e o *LocalDate* que representa o início do mês (2016-06-01), respectivamente:

```
LocalDateTime beginningOfDay = LocalDate.parse("2016-06-12").atStartOfDay();
LocalDate firstDayOfMonth = LocalDate.parse("2016-06-12")
  .with(TemporalAdjusters.firstDayOfMonth());

Cópia
```

Agora vamos dar uma olhada em como trabalhamos com a hora local.

3.2. Trabalhando com *o horário local*

LocalTimerepresenta *a***hora sem a data.**

Semelhante ao *LocalDate* , podemos criar uma instância de *LocalTime* a partir do relógio do sistema ou usando os métodos *parse* e *of* .

Vamos agora dar uma olhada rápida em algumas das APIs mais usadas.

É possível criaruma instância do *LocalTime atual a partir do relógio do sistema:*

```
LocalTime now = LocalTime.now();Cópia
```

Podemos criar um *LocalTime* representando 6h30 da manhã analisando uma representação em string:

```
LocalTime sixThirty = LocalTime.parse("06:30");Cópia
```

O método de fábrica *também* pode ser usado para criar um *LocalTime* . Este código cria *um LocalTime* representando 6h30 usando o método de fábrica:

```
LocalTime sixThirty = LocalTime.of(6, 30);Cópia
```

Vamos criar um *LocalTime* analisando uma string e adicionando uma hora a ela usando a API "plus". O resultado seria *um LocalTime* representando 7h30:

```
LocalTime sevenThirty = LocalTime.parse("06:30").plus(1,
ChronoUnit.HOURS);Cópia
```

Existem vários métodos getter disponíveis que podem ser usados para obter unidades de tempo específicas, como horas, minutos e segundos:

```
int six = LocalTime.parse("06:30").getHour();Cópia
```

Também podemos verificar se um horário específico é anterior ou posterior a outro horário específico. Este exemplo de código compara dois *horários LocalTime* para os quais o resultado seria verdadeiro:

```
boolean isbefore =
LocalTime.parse("06:30").isBefore(LocalTime.parse("07:30"));Cópia
```

Por fim, os horários máximo, mínimo e meio-dia de um dia podem ser obtidos por meio de constantes na classe *LocalTime* . Isso é muito útil ao realizar consultas em bancos de dados para encontrar registros dentro de um determinado período de tempo.

Por exemplo, o código abaixo representa 23:59:59.99:

```
LocalTime maxTime = LocalTime.MAXCópia
```

Agora vamos analisar o *LocalDateTime* .

3.3. Trabalhando com *LocalDateTime*

LocalDateTime é usado para representar **uma combinação de data e hora.** Esta é a classe mais comumente usada quando precisamos de uma combinação de data e hora.

A classe oferece uma variedade de APIs. Aqui, vamos analisar algumas das mais utilizadas.

Uma instância de *LocalDateTime* pode ser obtida a partir do relógio do sistema, de forma semelhante a *LocalDate* e *LocalTime* :

```
LocalDateTime.now();Cópia
```

Os exemplos de código abaixo explicam como criar uma instância usando os métodos de fábrica "of" e "parse". O resultado seria uma instância *de LocalDateTime* representando 20 de fevereiro de 2015, 6h30:

```
LocalDateTime.of(2015, Month.FEBRUARY, 20, 06, 30);Cópia
```

```
LocalDateTime.parse("2015-02-20T06:30:00");Cópia
```

Existem APIs utilitárias que permitem a adição e subtração de unidades de tempo específicas, como dias, meses, anos e minutos.

O código abaixo demonstra os métodos “plus” e “minus”. Essas APIs se comportam exatamente como suas contrapartes em *LocalDate* e *LocalTime* :

```
localDateTime.plusDays(1);Cópia
```

```
localDateTime.minusHours(2);Cópia
```

Métodos getter também estão disponíveis para extrair unidades específicas, semelhantes às classes de data e hora. Dado o exemplo de código acima , a seguinte *instrução* retornará o mês de fevereiro:

```
localDateTime.getMonth();Cópia
```

4. Usandoa API *ZonedDateTime*

Java 8 provides *ZonedDateTime* when we need to deal with time-zone-specific date and time. The *ZoneId* is an identifier used to represent different zones. There are about 40 different time zones, and the *ZoneId* represents them as follows.

Here, we create a *Zone* for Paris:

```
ZoneId zoneId = ZoneId.of("Europe/Paris");



Cópia
```

E podemos obter um conjunto com todos os IDs de zona:

```
Set<String> allZoneIds = ZoneId.getAvailableZoneIds();Cópia
```

O *LocalDateTime* pode ser convertido para um fuso horário específico:

```
ZonedDateTime zonedDateTime = ZonedDateTime.of(localDateTime, zoneId);Cópia
```

O *ZonedDateTime* fornece o método *parse* para obter a data e hora específicas do fuso horário:

```
ZonedDateTime.parse("2015-05-03T10:15:30+01:00[Europe/Paris]");Cópia
```

Outra forma de trabalhar com fusos horários é usando *o OffsetDateTime* . O *OffsetDateTime* é uma representação imutável de uma data e hora com um deslocamento. Essa classe armazena todos os campos de data e hora, com precisão de nanossegundos, bem como o deslocamento em relação ao UTC/Greenwich.

A instância *OffSetDateTime* pode ser criada usando *ZoneOffset* . Aqui, criamos um *LocalDateTime* representando 6h30 da manhã do dia 20 de fevereiro de 2015:

```
LocalDateTime localDateTime = LocalDateTime.of(2015, Month.FEBRUARY, 20, 06,
30);Cópia
```

Em seguida, adicionamos duas horas ao horário criando um *ZoneOffset* e definindo-o para a instância *localDateTime* :

```
ZoneOffset offset = ZoneOffset.of("+02:00");

OffsetDateTime offSetByTwo = OffsetDateTime
  .of(localDateTime, offset);

Cópia
```

Agora temos uma *data e hora local* de 2015-02-20 06:30 +02:00.

Agora vamos ver como modificar valores de data e hora usando asclasses *Period* e *Duration* .

5. Utilizando *Período* e *Duração*

A classe *Period* representa uma quantidade de tempo em termos de anos, meses e dias, e a classe *Duration* representa uma quantidade de tempo em termos de segundos e nanossegundos.

5.1. Trabalhando com *o período*

A classe *Period* é amplamente utilizada para modificar valores de uma data ou para obter a diferença entre duas datas:

```
LocalDate initialDate = LocalDate.parse("2007-05-10");Cópia
```

Podemos manipular a *data* usando *o ponto* :

```
LocalDate finalDate = initialDate.plus(Period.ofDays(5));Cópia
```

A classe *Period* possui vários métodos getter, como *getYears* , *getMonths* e *getDays,* para obter valores de umobjeto *Period* .

Por exemplo, isso retorna um valor *inteiro* de 5, pois estamos tentando obter a diferença em termos de dias *:*

```
int five = Period.between(initialDate, finalDate).getDays();Cópia
```

Podemos obter o *período* entre duas datas em uma unidade específica, como dias, meses ou anos, usando *ChronoUnit.between* :

```
long five = ChronoUnit.DAYS.between(initialDate, finalDate);Cópia
```

Este exemplo de código retorna cinco dias.

Vamos prosseguir analisando a classe *Duration* .

5.2. Trabalhando com *a duração*

Semelhante à *classe Period,* a *classe Duration* é usada para lidar com *o tempo.*

Vamos criar um *horário local* de 6:30 e, em seguida, adicionar uma duração de 30 segundos para obter o *horário local* de 6:30:30:

```
LocalTime initialTime = LocalTime.of(6, 30, 0);

LocalTime finalTime = initialTime.plus(Duration.ofSeconds(30));

Cópia
```

Podemos obter a *duração* entre dois instantes tanto como uma *duração* quanto como uma unidade específica.

Primeiro, usamos o método *between()* da classe *Duration* para encontrar a diferença de tempo entre *finalTime* e *initialTime* e retornar essa diferença em segundos:

```
long thirty = Duration.between(initialTime, finalTime).getSeconds();Cópia
```

No segundo exemplo, usamos o método *between()* da classe *ChronoUnit* para realizar a mesma operação:

```
long thirty = ChronoUnit.SECONDS.between(initialTime, finalTime);Cópia
```

Agora vamos ver como converter *a data* e *o calendário* existentes para a nova *data* / *hora* .

6. Compatibilidade com *data* e *calendário*

O Java 8 adicionou o método *toInstant()* , que ajuda a converter instâncias existentes *de Date* e *Calendar* para a nova API de Date e Time:

```
LocalDateTime.ofInstant(date.toInstant(), ZoneId.systemDefault());
LocalDateTime.ofInstant(calendar.toInstant(), ZoneId.systemDefault());

Cópia
```

O *LocalDateTime* pode ser construído a partir dos segundos da época Unix. O resultado do código abaixo seria um *LocalDateTime* representando 2016-06-13T11:34:50:

```
LocalDateTime.ofEpochSecond(1465817690, 0, ZoneOffset.UTC);Cópia
```

Agora vamos abordar a formatação *de data* e *hora* .

7.Formatação *de data* e *hora*

O Java 8 fornece APIs para facilitar a formatação de *data* e *hora* :

```
LocalDateTime localDateTime = LocalDateTime.of(2015, Month.JANUARY, 25, 6,
30);Cópia
```

Este código utiliza o formato de data ISO para formatar a data local, resultando em 2015-01-25:

```
String localDateString =
localDateTime.format(DateTimeFormatter.ISO_DATE);Cópia
```

O *DateTimeFormatter* oferece diversas opções de formatação padrão.

Também é possível fornecer padrões personalizados ao método format, que neste caso retorna um *LocalDate* como 2015/01/25:

```
localDateTime.format(DateTimeFormatter.ofPattern("yyyy/MM/dd"));Cópia
```

Podemos definir o estilo de formatação como *CURTO* , *LONGO* ou *MÉDIO* como parte da opção de formatação.

Por exemplo, isso resultaria em uma saída representando *LocalDateTime* em 25 de janeiro de 2015, às 06:30:00:

```
localDateTime
  .format(DateTimeFormatter.ofLocalizedDateTime(FormatStyle.MEDIUM)
  .withLocale(Locale.UK));
```

```
Cópia
```

Vamos dar uma olhada nas alternativas disponíveis para as APIs *de data / hora* do Java 8 Core .

8. Opções alternativas e de portabilidade reversa

8.1. Utilizando o Projeto ThreeTen

Para organizações que estão em processo de migração do Java 7 ou Java 6 para o Java 8 e que desejam usar a API de data e hora, o projeto ThreeTen oferece a funcionalidade de retrocompatibilidade.

Os desenvolvedores podem usar as classes disponíveis neste projeto para obter a mesma funcionalidade das novas APIs *de Data* e *Hora* do Java 8. E, ao migrarem para o Java 8, os pacotes podem ser trocados.

O artefato do projeto ThreeTen pode ser encontrado no repositório Maven Central :

```
<dependency>
    <groupId>org.threeten</groupId>
    <artifactId>threetenbp</artifactId>
    <version>1.3.1</version>
</dependency>

Cópia
```

8.2. Biblioteca Joda-Time

Outra alternativa à biblioteca *de data* e *hora* do Java 8 é a biblioteca Joda-Time . De fato, a API *de data / hora* do Java 8 foi desenvolvida em conjunto pelo autor da biblioteca Joda-Time (Stephen Colebourne) e pela Oracle. Essa biblioteca oferece praticamente todas as funcionalidades suportadas peloprojeto *de data / hora do Java 8.*

O artefato pode ser encontrado no Maven Central incluindo a seguinte dependência pom em nosso projeto:

```
<dependency>
    <groupId>joda-time</groupId>
    <artifactId>joda-time</artifactId>
    <version>2.9.4</version>
</dependency>


Cópia
```

9. Conclusão

O Java 8 oferece um conjunto abrangente de APIs com um design consistente para facilitar o desenvolvimento.

O código que fundamenta este artigo está disponível no GitHub. Após fazer **login como membro Baeldung Pro** , comece a aprender e a programar no projeto.